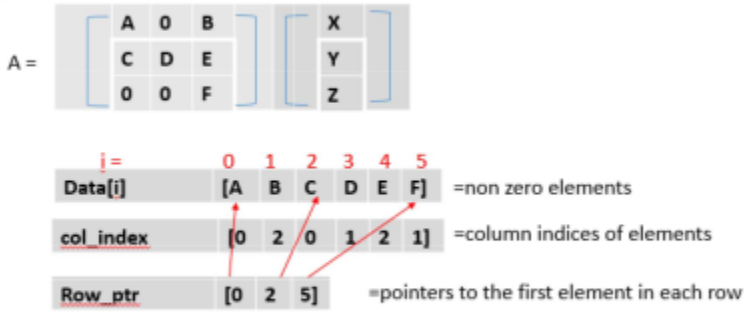
PPL LAB 9 230905290 Roll no 32
Rohan Shenoy CSE D1

Q1]

Lab Exercises:



1. Write a program in CUDA to perform parallel Sparse Matrix - Vector multiplication using compressed sparse row (CSR) storage format. Represent the input sparse matrix in CSR format in the host code.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void spmv_csr_kernel(int num_rows, const float* data, const
int* col_index, const int* row_ptr, const float* x, float* y) {
   int row = blockIdx.x * blockDim.x + threadIdx.x;

   if (row < num_rows) {
       float dot_product = 0.0f;
       int row_start = row_ptr[row];
       int row_end = row_ptr[row + 1];

       for (int i = row_start; i < row_end; i++) {
           dot_product += data[i] * x[col_index[i]];
       }
```

```
        y[row] = dot_product;
    }
}

int main() {
    const int num_rows = 3;
    const int num_cols = 3;
    const int nnz = 6;

    float h_data[] = {10.0f, 20.0f, 30.0f, 40.0f, 50.0f, 60.0f};
    int h_col_index[] = {0, 2, 0, 1, 2, 2};
    int h_row_ptr[] = {0, 2, 5, 6};

    float h_x[] = {1.0f, 2.0f, 3.0f};
    float h_y[3] = {0.0f};


    float *d_data, *d_x, *d_y;
    int *d_col_index, *d_row_ptr;

    cudaMalloc((void**)&d_data, nnz * sizeof(float));
    cudaMalloc((void**)&d_col_index, nnz * sizeof(int));
    cudaMalloc((void**)&d_row_ptr, (num_rows + 1) * sizeof(int));
    cudaMalloc((void**)&d_x, num_cols * sizeof(float));
    cudaMalloc((void**)&d_y, num_rows * sizeof(float));


    cudaMemcpy(d_data, h_data, nnz * sizeof(float),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_col_index, h_col_index, nnz * sizeof(int),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_row_ptr, h_row_ptr, (num_rows + 1) * sizeof(int),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_x, h_x, num_cols * sizeof(float), cudaMemcpyHostToDevice);

    int blockSize = 256;
    int gridSize = (num_rows + blockSize - 1) / blockSize;

    spmv_csr_kernel<<<gridSize, blockSize>>>(num_rows, d_data, d_col_index,
d_row_ptr, d_x, d_y);
```

```
    cudaDeviceSynchronize();

    cudaMemcpy(h_y, d_y, num_rows * sizeof(float), cudaMemcpyDeviceToHost);

    printf("Result Vector Y:\n");
    for (int i = 0; i < num_rows; i++) {
        printf("Y[%d] = %.2f\n", i, h_y[i]);
    }

    cudaFree(d_data);
    cudaFree(d_col_index);
    cudaFree(d_row_ptr);
    cudaFree(d_x);
    cudaFree(d_y);

    return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab9$ nvcc q1.cu
6CSED1@debian:~/Desktop/ppl_lab/lab9$ ./a.out
Result Vector Y:
Y[0] = 70.00
Y[1] = 260.00
Y[2] = 180.00
6CSED1@debian:~/Desktop/ppl_lab/lab9$
```

Q2]

2. Write a program in CUDA to read MXN matrix A and replace 1st row of this matrix by same elements, 2nd row elements by square of each element and 3rd row elements by cube of each element and so on.

CODE]

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda_runtime.h>

__global__ void transformMatrixKernel(float* matrix, int M, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        int index = row * N + col;

        float power = (float)(row + 1);
        matrix[index] = powf(matrix[index], power);
    }
}

int main() {
    int M = 4;
    int N = 3;
    size_t size = M * N * sizeof(float);


    float* h_A = (float*)malloc(size);

    printf("Original Matrix:\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            h_A[i * N + j] = (float)(j + 2);
            printf("%.1f\t", h_A[i * N + j]);
        }
        printf("\n");
    }

    float* d_A;
    cudaMalloc((void**)&d_A, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
```

```
    dim3 threadsPerBlock(16, 16);
    dim3 blocksPerGrid((N + threadsPerBlock.x - 1) / threadsPerBlock.x,
                       (M + threadsPerBlock.y - 1) / threadsPerBlock.y);

    transformMatrixKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, M, N);

    cudaDeviceSynchronize();

    cudaMemcpy(h_A, d_A, size, cudaMemcpyDeviceToHost);

    printf("\nTransformed Matrix (Row 1: same, Row 2: sq, Row 3:
cube...):\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            printf("%.1f\t", h_A[i * N + j]);
        }
        printf("\n");
    }

    cudaFree(d_A);
    free(h_A);

    return 0;
}
```

CODE]

```
6CSED1@debian:~/Desktop/ppl_lab/lab9$ nvcc q2.cu
6CSED1@debian:~/Desktop/ppl_lab/lab9$ ./a.out
Original Matrix:
2.0     3.0     4.0
2.0     3.0     4.0
2.0     3.0     4.0
2.0     3.0     4.0

Transformed Matrix (Row 1: same, Row 2: sq, Row 3: cube...):
2.0     3.0     4.0
4.0     9.0     16.0
8.0     27.0    64.0
16.0    81.0    256.0
```

Q3]

3. Write a CUDA program that reads a matrix A of size MXN and produce an output matrix B of same size such that it replaces all the non-border elements(numbers in bold) of A with its equivalent 1's complement and remaining elements same as matrix A.

A

1	2	3	4
6	**5**	**8**	3
2	**4**	**10**	1
9	1	2	5

B

1	2	3	4
6	**10**	**111**	3
2	**11**	**101**	1
9	1	2	5

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void complementKernel(int *A, int *B, int M, int N) {
   int r = blockIdx.y * blockDim.y + threadIdx.y;
   int c = blockIdx.x * blockDim.x + threadIdx.x;

   if (r < M && c < N) {
       int idx = r * N + c;

       if (r > 0 && r < M - 1 && c > 0 && c < N - 1) {
           int val = A[idx];

           int temp = val;
           int mask = 0;
           if (val == 0) mask = 1;
           while (temp > 0) {
               mask = (mask << 1) | 1;
               temp >>= 1;
           }
           int complement = val ^ mask;

           int binary_as_int = 0;
           int base = 1;
           while (complement > 0) {
               binary_as_int += (complement % 2) * base;
               complement /= 2;
               base *= 10;
```

```
            }
            B[idx] = binary_as_int;
        }
        else {
            B[idx] = A[idx];
        }
    }
}

int main() {
    int M = 4, N = 4;
    size_t size = M * N * sizeof(int);

    int h_A[] = {
        1, 2, 3, 4,
        6, 5, 8, 3,
        2, 4, 10, 1,
        9, 1, 2, 5
    };
    int h_B[16];

    int *d_A, *d_B;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

    dim3 block(16, 16);
    dim3 grid((N + block.x - 1) / block.x, (M + block.y - 1) / block.y);

    complementKernel<<<grid, block>>>(d_A, d_B, M, N);

    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);

    printf("Matrix B:\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", h_B[i * N + j]);
        }
        printf("\n");
```

```
    }

    cudaFree(d_A);
    cudaFree(d_B);
    return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab9$ nvcc q3.cu
6CSED1@debian:~/Desktop/ppl_lab/lab9$ ./a.out
Matrix B:
1       2       3       4
6       10      111     3
2       11      101     1
9       1       2       5
```

ADDITIONAL QUESTIONS

AQ1]

1. Write a CUDA program which reads an input matrix A of size MXN and produces an output matrix B of size MXN such that, each element of the output matrix is calculated in parallel. Each element, B[i][j], in the output matrix is obtained by adding the elements in i^{th} row and j^{th} column of the input matrix A.

Example:

A

1	**2**	**3**
4	5	6

O/p:

B

11	13	15
20	22	24

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void rowColSumKernel(int *A, int *B, int M, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    if (row < M && col < N) {
        int rowSum = 0;
        int colSum = 0;


        for (int k = 0; k < N; k++) {
            rowSum += A[row * N + k];
        }


        for (int k = 0; k < M; k++) {
            colSum += A[k * N + col];
        }


        B[row * N + col] = rowSum + colSum;
    }
}

int main() {
    int M = 2;
    int N = 3;
    int size = M * N * sizeof(int);

    int *h_A = (int*)malloc(size);
    int *h_B = (int*)malloc(size);

    int values[] = {1, 2, 3, 4, 5, 6};
    for (int i = 0; i < M * N; i++) h_A[i] = values[i];

    int *d_A, *d_B;
    cudaMalloc((void**)&d_A, size);
    cudaMalloc((void**)&d_B, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

    dim3 threadsPerBlock(16, 16);
    dim3 blocksPerGrid((N + threadsPerBlock.x - 1) / threadsPerBlock.x,
                       (M + threadsPerBlock.y - 1) / threadsPerBlock.y);
```

```
    rowColSumKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, M, N);

    cudaMemcpy(h_B, d_B, size, cudaMemcpyDeviceToHost);

    printf("Output Matrix B:\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d\t", h_B[i * N + j]);
        }
        printf("\n");
    }

    cudaFree(d_A);
    cudaFree(d_B);
    free(h_A);
    free(h_B);

    return 0;
}
```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab9$ nvcc aq1.cu
6CSED1@debian:~/Desktop/ppl_lab/lab9$ ./a.out
Output Matrix B:
11      13      15
20      22      24
```

AQ2]

2. Write a CUDA program that reads a character type matrix A and integer type matrix B of size MXN. It produces an output string *STR* such that, every character of *A* is repeated *r* times (where *r* is the integer

value in matrix B which is having the same index as that of the character taken in A). Write the kernel such that every value of input matrix must be produced required number of times by one thread.

Example:

A				B			
p	C	a	P	1	2	4	3
e	X	a	M	2	4	3	2

Output String STR: pCCaaaaPPPeeXXXXaaaMM

CODE]

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void repeatCharsKernel(char *d_A, int *d_B, int *d_offsets,
char *d_STR, int totalElements) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < totalElements) {
        char c = d_A[idx];
        int count = d_B[idx];
        int startPos = d_offsets[idx];

        for (int i = 0; i < count; i++) {
            d_STR[startPos + i] = c;
        }
    }
}

int main() {
    int M = 2, N = 4;
    int totalElements = M * N;
```

```
    char h_A[] = {'p', 'C', 'a', 'P', 'e', 'X', 'a', 'M'};
    int h_B[] = {1, 2, 4, 3, 2, 4, 3, 2};

    int *h_offsets = (int*)malloc(totalElements * sizeof(int));
    int currentOffset = 0;
    for (int i = 0; i < totalElements; i++) {
        h_offsets[i] = currentOffset;
        currentOffset += h_B[i];
    }
    int totalStringLength = currentOffset;

    char *d_A, *d_STR;
    int *d_B, *d_offsets;

    cudaMalloc((void**)&d_A, totalElements * sizeof(char));
    cudaMalloc((void**)&d_B, totalElements * sizeof(int));
    cudaMalloc((void**)&d_offsets, totalElements * sizeof(int));
    cudaMalloc((void**)&d_STR, (totalStringLength + 1) * sizeof(char));

    cudaMemcpy(d_A, h_A, totalElements * sizeof(char),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, totalElements * sizeof(int),
cudaMemcpyHostToDevice);
    cudaMemcpy(d_offsets, h_offsets, totalElements * sizeof(int),
cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (totalElements + threadsPerBlock - 1) /
threadsPerBlock;

    repeatCharsKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B,
d_offsets, d_STR, totalElements);

    char *h_STR = (char*)malloc((totalStringLength + 1) * sizeof(char));
    cudaMemcpy(h_STR, d_STR, totalStringLength * sizeof(char),
cudaMemcpyDeviceToHost);
    h_STR[totalStringLength] = '\0';

    printf("Output String STR: %s\n", h_STR);
```

```
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_offsets); cudaFree(d_STR);
    free(h_offsets); free(h_STR);

    return 0;
}
```

OUTPUT]